研艺士
YANZHISHI

提高篇 （3）——数据结构专题

主讲人：灵灵老师

计算机考研

目录

1. 并查集
2. 图的相关概念
3. 图的遍历
4. 最小生成树
5. 最短路径
6. 拓扑排序

研之士 YANZHISHI

并查集

1. 定义概述：

并查集是一种数据结构，用于处理一组不相交的**集合**的**合并与查询**操作。

2. 核心操作：

① 查找findFather(x)：查找元素x所属的集合（即确定元素所属的根节点）。

② 合并Union(x, y)：将包含元素x和y的两个集合合并成一个集合。

3. 应用场景：（连通性问题）

图的连通分量：在无向图中，判断两个节点是否属于同一个连通分量。

Kruskal 算法：在最小生成树算法中，用于判断是否形成环路。

4.并查集的基本操作与实现：

那么并查集是用什么实现的呢？其实就是用一个数组：

```
int father[N];
```

其中 fahter[i]表示元素 i 的父亲结点，而父亲结点本身也是这个集合内的元素（1≤i≤N）。例如 father[1] = 2 就表示元素 1 的父亲结点是元素 2，以这种父系关系来表示元素所属的集合。另外，如果 father[i] == i，则说明元素 i 是该集合的根结点，但对同一个集合来说只存在一个根结点，且将其作为所属集合的标识。

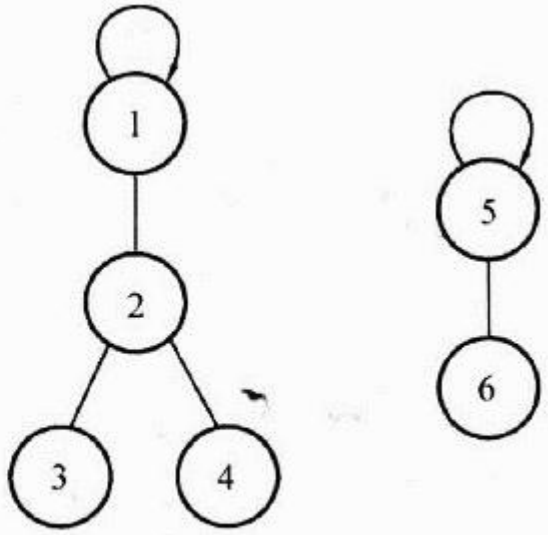


图 9-37　并查集父亲关系示意图

4.并查集的基本操作与实现：

（1）初始化

每个元素自成一个集合，即每个元素的父节点初始化为它本身。

```
for(int i = 1; i <= N; i++) {
    father[i] = i;  //令 father[i]为-1 也可，此处以 father[i] = i 为例
}
```

（2）查找

确定一个元素属于哪一个集合，通过不断查找其父节点直到找到根节点（**每个集合只有一个根节点**）。

```
int findFather(int x) {
    if(x == father[x]) return x;  //如果找到根结点，则返回根结点编号 x
    else return findFather(father[x]);  //否则，递归判断 x 的父亲结点是否是根结点
}
```

并查集

（3）合并

合并是指把两个集合合并成一个集合。

先判断两个元素是否属于同一个集合，只有当两个元素属于不同集合时才合并。

合并：把其中一个集合的根结点的父亲指向另一个集合的根结点。

```
void Union(int a, int b) {
    int faA = findFather(a);  //查找 a 的根结点，记为 faA
    int faB = findFather(b);  //查找 b 的根结点，记为 faB
    if(faA != faB) {          //如果不属于同一个集合
        father[faA] = faB;    //合并它们
}
}
```

在合并的过程中，只对两个不同的集合进行合并。
这就保证了在同一个集合中一定不会产生环。

图的相关概念

图的定义

二元组G=(V,E)，两个集合：

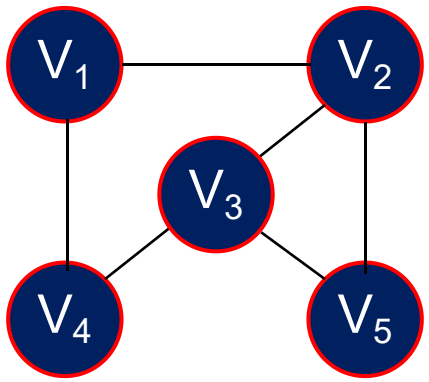
V(vertex)：顶点的有穷非空集合；

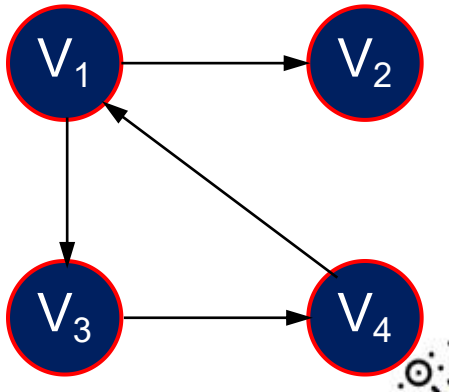
E(edge)：边(或弧)的有穷集合，若E为空，只有顶点而没有边。

一个顶点就是一个数据元素，顶点取值可以是一个字符、一个字符串或一个整数。

有向图：每条边均有方向，用有序偶表示。

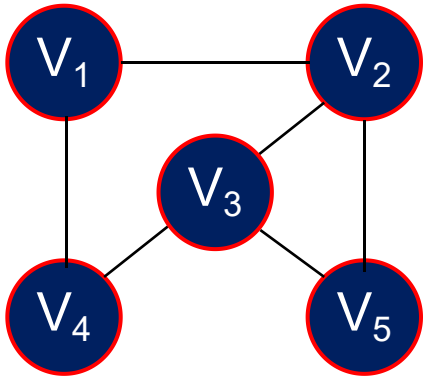
无向图：每条边均无方向，用无序偶表示。





图的相关概念

基本术语



完全图：任意两个顶点都有边相连，无向图一条边相连，有向图两条边相连。

稀疏图：边或弧很少的图。

稠密图：边或弧较多的图，如完全图。

邻接点：两个顶点有边或弧相连，则互为邻接点。

路径：从一个顶点到另一个顶点所经过的顶点序列。

顶点的度：与某个顶点相关联的边的数目（有向树：只有1个顶点的入度为0,其余顶点的入度均为1）。

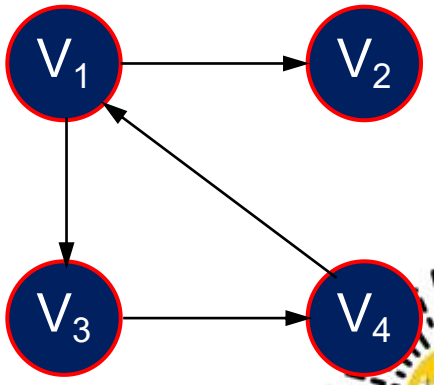
路径长度：路径上边或弧的数目（有时是权值之和）。

连通图：在无向图中，任意两个顶点均有路径。

极小连通子图：图中的连通子图，删除任何一条边，则该子图不再连通。

连通图的生成树：极小连通子图，包含无向图的所有顶点。

（包含n个顶点和n-1条边）



图的相关概念

图的存储

（1）邻接矩阵（二维数组）

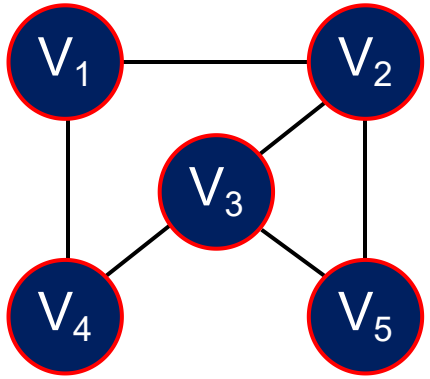
设图的顶点标号为0,1,...,N-1，那么可以令二维数组G[N][N]的两维分别表示图的顶点标号。

如果 G[i][j]为1，则说明顶点i和顶点j之间有边，

如果G[i][j]为0，则说明顶点i和顶点j之间不存在边，

而这个二维数则被称为邻接矩阵。

如果存在边权，则可以令 G[i][j]存放边权。



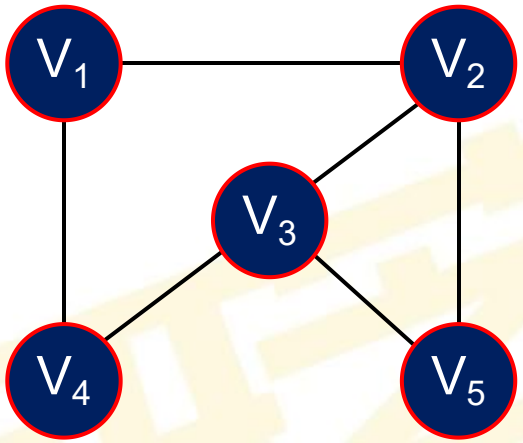
	V1	V2	V3	V4	V5
V1	0	1	0	1	0
V2	1	0	1	0	1
V3	0	1	0	1	1
V4	1	0	1	0	0
v5	0	1	1	0	0

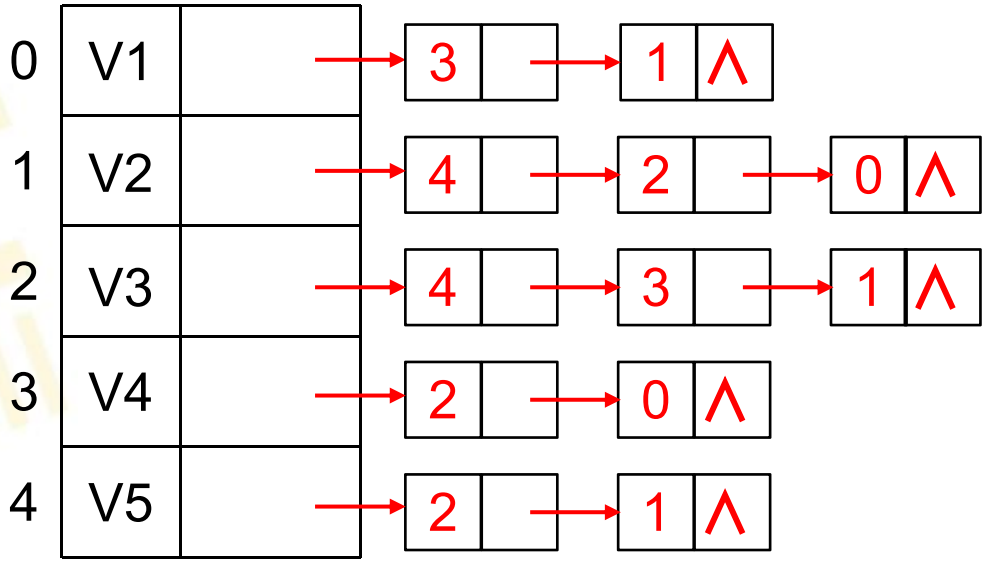
图的存储

（2）邻接表（顺序+链式）

顶点表：通常是一个数组，用于存放图的顶点信息。数组中的每个元素对应图中的一个顶点，每个元素内部包含顶点的数据和指向该顶点邻接点的链表的头指针。

边表：由链表组成（把同一个顶点的所有出边放在一个链表中）。





图的相关概念

邻接表（使用vector实现）

由于 vector 有变长数组之称，因此可以开一个 vector 数组 Adj[N]，其中N为顶点个数。这样每个Adj[i]就都是一个变长数组 vector，使得存储空间只与图的边数有关。如果邻接表只存放每条边的终点编号，而不存放边权，则vector中的元素类型可以直接定义为 int 型，如下所示:

```
vector<int> Adj[N];
```

如果想添加一条从x号顶点到达y号顶点的有向边:只需要在 Adj[x]中添加终点编号y即可,代码如下所示(如果是无向边，就再添加一条从y号顶点到达x号顶点的有向边):

```
Adj[x].push back(y);
Adj[y].push back(x);
```

图的遍历

遍历定义：

从图中任一顶点出发，按照某种方法，访问图中所有顶点，且每个顶点只访问一次。遍历是图的最重要的基本运算，也其它运算的基础。 遍历实质是寻找邻接顶点，顶点之间的邻接关系显得十分重要。

关键问题：

图中可能存在回路，任一顶点都可能与其它顶点连通，在访问某个顶点后，可能会沿着某些边又回到了曾经访问过的顶点。

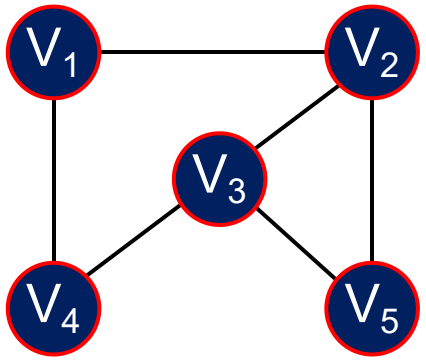
解决办法：

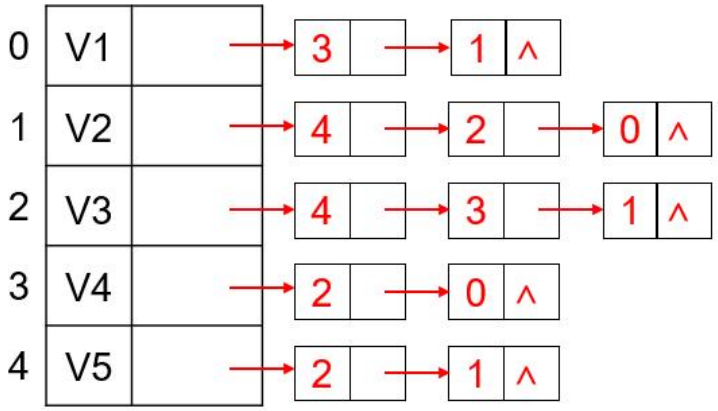
为避免重复访问同一顶点，设置一个辅助数组visited[n]，用来记录顶点是否已经访问。辅助数组visited的全部元素的初值设置为0(false)；如果已经访问第i个顶点，则visited[i]设置为1(true)。

图的遍历

深度优先搜索遍历：

访问第一个未访问的邻接点，继续向前搜索，再访问第一未访问的邻接点。



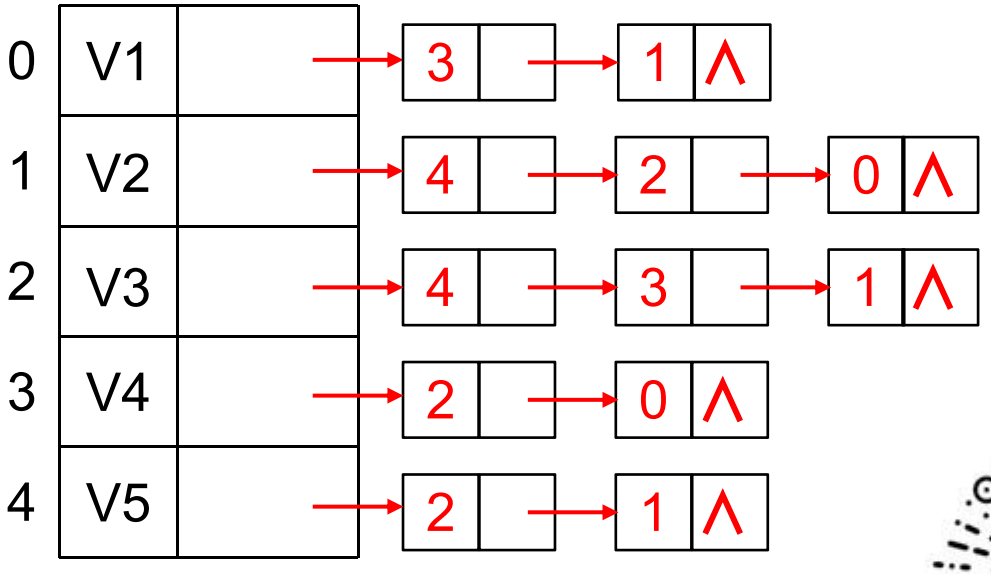


广度优先搜索：

访问当前顶点全部邻接点，向前搜索，再访问全部邻接点。

图的遍历

```
// 深度优先搜索函数
void dfs(int u) {
    visited[u] = true;      // 标记当前顶点为已访问
    for (从u出发能到达的所有顶点v) {      // 遍历当前顶点的所有邻接点
        if (!visited[v]) {      // 如果邻接点未被访问过
            dfs(v);      // 递归访问该邻接点
        }
    }
}
```



研之士 YANZHISHI

图的遍历

3587. 连通图_AcWing题库

研学士 YANZHISHI

最小生成树

最小生成树：

一个无向网(带有边权)可能有多棵**生成树**， 各边**权之和最小的生成树**称为图的最小生成树。

生成树：包含图中所有顶点的极小连通子图（n-1条边）。

极小连通子图：该子图是G 的连通子图，在该子图中删除任何一条边，子图不再连通。

最小生成树的应用：

在n个城市之间建立通信网（电力网，交通网），可供选择的线路总共有n(n-1)/2条，从中选择n-1条线路使n个城市连通。

问题：应当如何选择n-1条线路使成本最低?

回答：最小生成树!

最小生成树- Kruskal（克鲁斯卡尔）

kruskal 算法的基本思想为：在初始状态时隐去图中的所有边，这样图中每个顶点都自成一个连通块。之后执行下面的步骤：

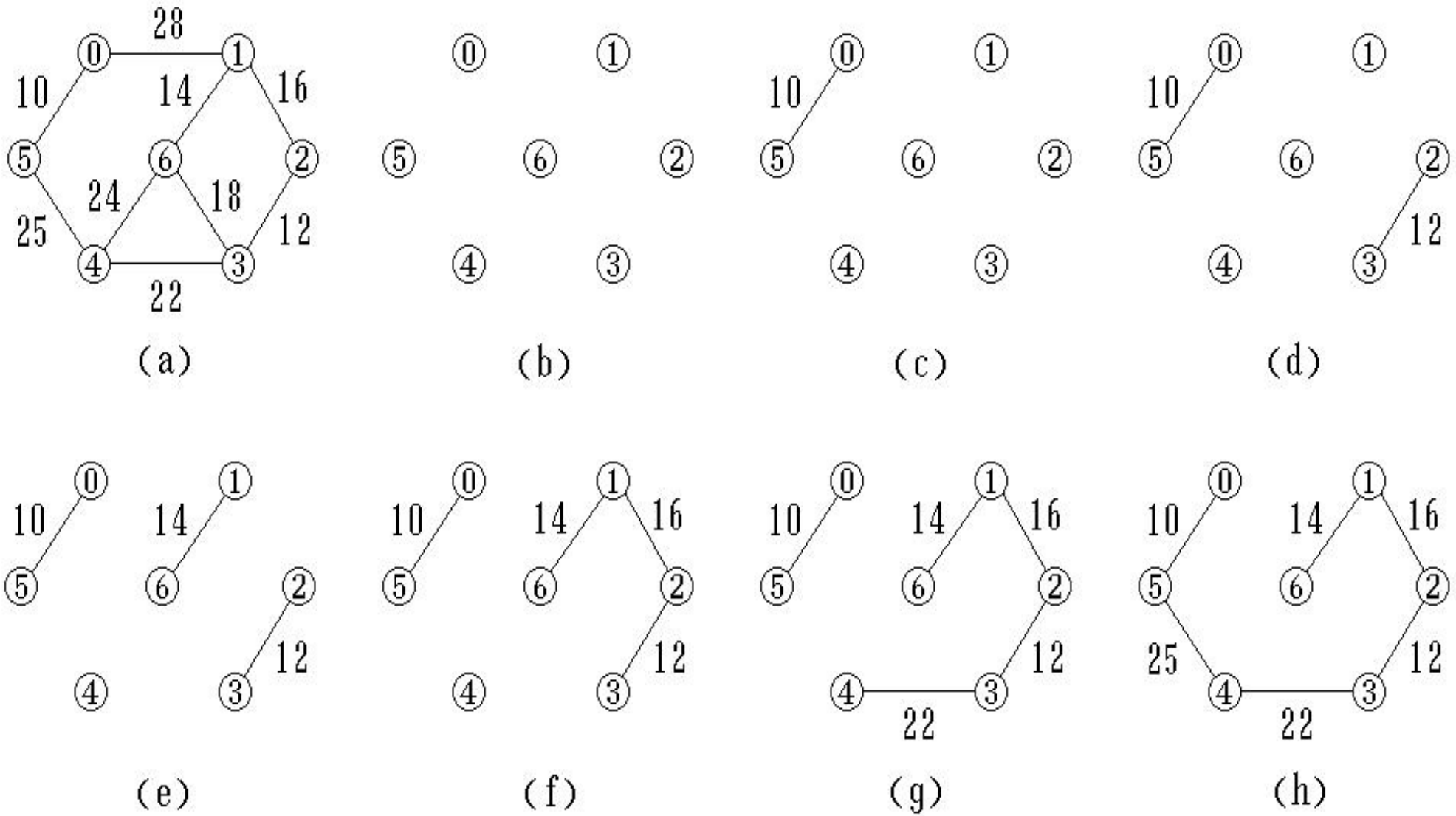
① 对所有边按边权从小到大进行排序。

② 按边权从小到大测试所有边，如果当前测试边所连接的两个顶点不在同一个连通块中，则把这条测试边加入当前最小生成树中；否则，将边舍弃。

③ 执行步骤②，直到最小生成树中的边数等于总顶点数减 1 或是测试完所有边时结束。而当结束时如果最小生成树的边数小于总顶点数减 1，说明该图不连通。

YANZHISHI

最小生成树- Kruskal（克鲁斯卡尔）



最小生成树

3647. 惠民工程_AcWing题库

最短路径

最短路径是图论中一个很经典的问题:

给定图G=(V，E)，求一条从起点到终点的路径，使得这条路径上经过的所有边的边权之和最小。

（注：最短路径与最小生成树不同，路径上不一定包含n个顶点）

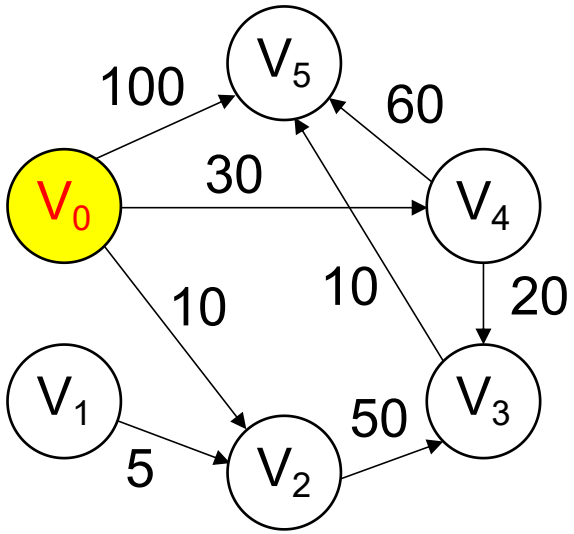
从顶点V_0到顶点V_5有路径：

0→5(路径长度100)

0→4→5(路径长度90)

0→4→3→5(路径长度60)

0→2→3→5(70)



最短路径-Dijkstra算法

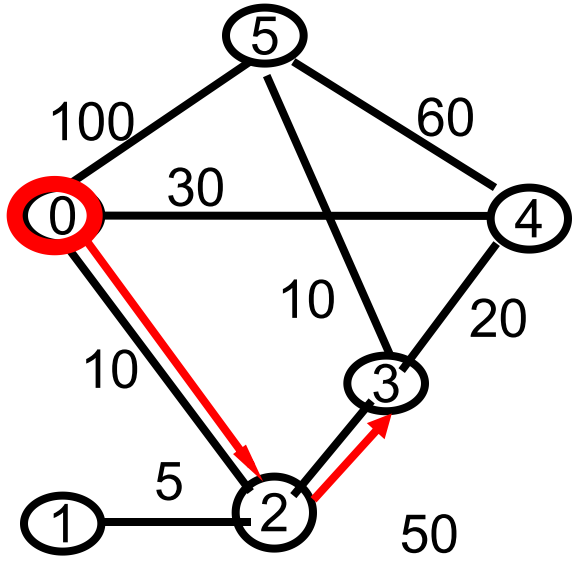
1.初始化：先找出从源点v_0到各终点v_k的直达路径（v_0,v_k），即通过一条弧到达的路径。

2.选择：从这些路径中找出一条长度最短的路径（v_0,u）。

3.更新：然后对其余各条路径进行适当调整：

若在图中存在弧（u,v_k），且（v_0,u）+（u,v_k）<（v_0,v_k），则以路径（v_0,u,v_k）代替（v_0,v_k）。

在调整后的各条路径中，再找长度最短的路径，依此类推。



拓扑排序

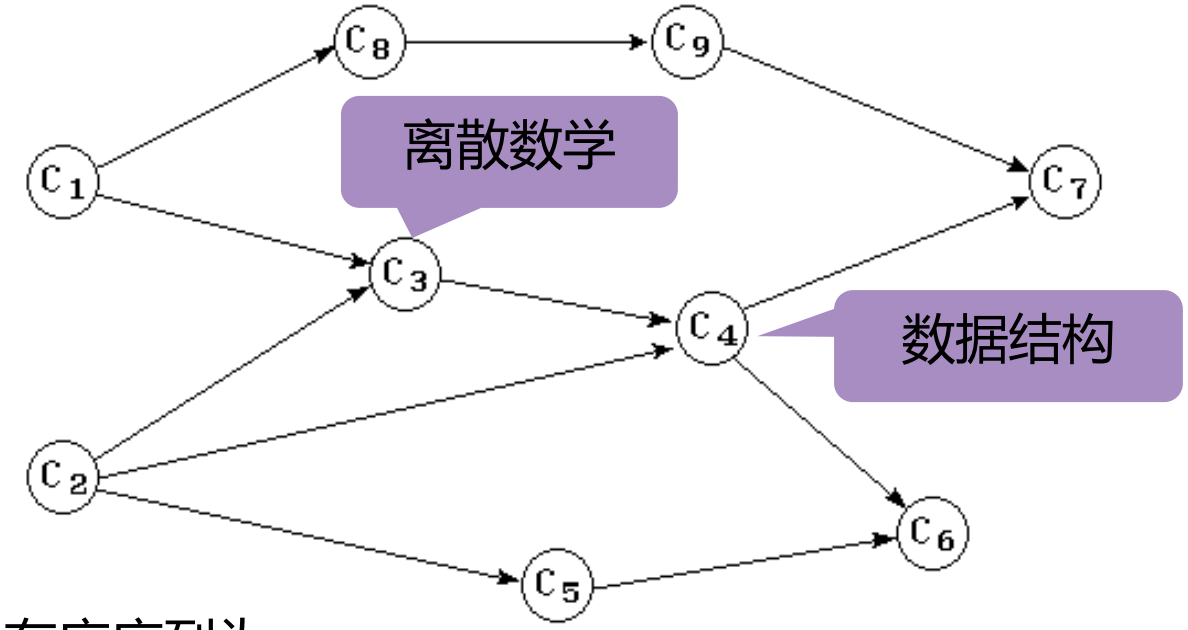
如果一个有向图的任意顶点都无法通过一些有向边回到自身，那么称这个有向图为**有向无环图。**

拓扑排序是将有向无环图G的所有顶点排成一个线性序列。

应用：

用有向图来描述一个工程或系统的进行过程。

比如教学计划的制定。哪些课程是必须先修的，哪些课程是可以并行学习的。



- 对学生选课工程图进行拓扑排序，得到的拓扑有序序列为

C_1 , C_2 , C_3 , C_4 , C_5 , C_6 , C_8 , C_9 , C_7

或 C_1 , C_8 , C_9 , C_2 , C_5 , C_3 , C_4 , C_7 , C_6

拓扑排序

01 输入有向无环图。

02 选一个没有直接前驱的顶点（**入度为0的点**），并输出之;

03 从图中删去该顶点, 同时删去所有它发出的有向边;

04 重复以上 2、3 步, 直到:

- ✓ 全部顶点均已输出，拓扑有序序列形成，拓扑排序完成；或:
- ✓ 图中还有未输出的顶点，但已跳出处理循环。这说明图中还剩下一些顶点，它们都有直接前驱，再也找不到没有前驱的顶点了。这时AOV网络中必定存在有向环。

GHZ YANZHISHI

拓扑排序

3704. 排队_AcWing题库

课后习题

KY268 第一题

3385. 玛雅人的密码

3675. 逃离迷宫

KY118 继续畅通工程

KY152 最短路径问题

研之士 YANZHISHI

谢谢聆听！